

Lesson 1: Introduction to Laravel and Environment Setup

□ Table of Contents

1. [What is Laravel?](#)
 2. [Why Choose Laravel?](#)
 3. [Laravel Project Structure](#)
 4. [Your First Laravel Application](#)
 5. [Understanding Request Lifecycle](#)
 6. [Practical Exercises](#)
-

What is Laravel?

Laravel is a **modern PHP web application framework** with an elegant and expressive syntax. Created by Taylor Otwell in 2011, it has become one of the most popular PHP frameworks in the world.

Key Features:

- **Elegant Syntax:** Clean, readable, and enjoyable code to write
- **MVC Architecture:** Model-View-Controller pattern for organized code
- **Built-in Tools:** Authentication, routing, sessions, caching ready to use
- **Eloquent ORM:** Beautiful database interactions
- **Blade Templates:** Powerful templating engine

- **Artisan Tool:** Command-line tool for common tasks
 - **Rich Ecosystem:** Packages like Sanctum, Horizon, Telescope, etc.
-

Why Choose Laravel?

1. Developer Experience

- Clear and comprehensive documentation
- Intuitive and expressive syntax
- Large and supportive community

2. Security

- Built-in SQL Injection protection
- CSRF (Cross-Site Request Forgery) protection
- XSS (Cross-Site Scripting) prevention
- Secure password encryption

3. Performance

- Built-in caching mechanisms
- Query optimization tools
- Queues for heavy tasks

4. Scalability

- Used by small startups to large enterprises

- Microservices support
- Horizontal scaling capabilities

Laravel Project Structure

When you look at a Laravel project, you'll see this structure:

```
laravel-project/
├── app/                # Core application code
│   ├── Http/
│   │   ├── Controllers/ # Controller classes
│   │   └── Middleware/  # Middleware
│   ├── Models/         # Eloquent models
│   └── Providers/       # Service providers
├── bootstrap/          # Framework bootstrap files
├── config/             # Configuration files
├── database/
│   ├── migrations/     # Database migrations
│   ├── seeders/        # Database seeders
│   └── factories/      # Model factories
├── public/             # Public files
│   └── index.php        # Entry point
├── resources/
│   ├── views/          # Blade templates
│   ├── css/            # CSS files
│   └── js/             # JavaScript files
├── routes/
│   ├── web.php         # Web routes
│   ├── api.php         # API routes
│   └── console.php     # Console commands
├── storage/            # Logs, cache, uploaded files
├── tests/              # Automated tests
├── vendor/             # Composer dependencies
├── .env               # Environment variables
├── artisan            # Artisan CLI tool
└── composer.json      # PHP dependencies
```

Important Folders Explained:

Folder	Purpose
app/	Contains core application code (models, controllers, middleware)
config/	All configuration files (database, mail, cache, etc.)
database/	Database migrations, seeders, and factories
public/	Entry point for all requests, contains public files
resources/views/	Blade template files
routes/	Define all application endpoints
storage/	Compiled Blade templates, uploaded files, logs, cache

Your First Laravel Application

Step 1: Check Your Environment

Make sure you have:

- PHP $\geq$ 8.1
- Composer (PHP package manager)
- MySQL/PostgreSQL/SQLite (database)

Check versions:

```
php -v
composer -v
```

Step 2: Understanding the Current Project

You're now in a Laravel project! Let's explore what's there.

Entry Point

Every request goes through `public/index.php`, which:

1. Loads the autoloader
2. Bootstraps the application
3. Handles the request
4. Returns the response

Configuration

The `.env` file contains environment-specific settings:

```
APP_NAME=Laravel
APP_ENV=local
APP_DEBUG=true
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
```

Important: Never commit `.env` to version control! It contains sensitive data.

Step 3: Artisan - Your Best Friend

Laravel comes with **Artisan**, a powerful command-line tool.

Try these commands:

```
# List all available commands
php artisan list

# Check Laravel version
php artisan --version

# Display routes
php artisan route:list
```

```
# Start development server
php artisan serve
```

The `php artisan serve` command starts a local development server at `http://localhost:8000`.

Understanding Request Lifecycle

This is what happens when a user visits your Laravel application:

```
1. User Request
  ↓
2. public/index.php (Entry point)
  ↓
3. Bootstrap Laravel
  ↓
4. Load Service Providers
  ↓
5. Routing (routes/web.php or routes/api.php)
  ↓
6. Middleware (Security, authentication, etc.)
  ↓
7. Controller (Business logic)
  ↓
8. Model (Database interaction) - if needed
  ↓
9. View (Blade template) - for web requests
  ↓
10. Send Response to User
```

Example Flow:

User visits: `http://yourapp.com/welcome`

1. **Route** (`routes/web.php`):

```
Route::get('/welcome', function () {
    return view('welcome');
});
```

```
});
```

2. **View** (`resources/views/welcome.blade.php`):

```
<!DOCTYPE html>
<html>
<head>
<title>Welcome</title>
</head>
<body>
<h1>Welcome to Laravel!</h1>
</body>
</html>
```

3. **Response**: HTML is rendered and sent to the browser

Practical Exercises

Exercise 1: Explore Your Project

Open your Laravel project and locate:

- [] Where is the main entry point? (Hint: `public/index.php`)
- [] Where are routes defined? (Hint: `routes/web.php`)
- [] Where are views stored? (Hint: `resources/views/`)
- [] What's in the `.env` file?

Exercise 2: Start Development Server

```
php artisan serve
```

Visit `http://localhost:8000` in your browser. What do you see?

Exercise 3: Create Your First Route

1. Open `routes/web.php`
2. Add this new route:

```
Route::get('/hello', function () {  
    return '<h1>Hello, I\'m learning Laravel!</h1>';  
});
```

3. Visit `http://localhost:8000/hello`

Challenge: Create routes for:

- `/about` - returns an "About" page
- `/contact` - returns a "Contact" page

Exercise 4: Explore Artisan Commands

Execute these commands and observe the results:

```
# List all routes  
php artisan route:list  
  
# Clear cache  
php artisan cache:clear  
  
# Display all artisan commands  
php artisan list
```

Exercise 5: Create a Simple View

1. Create a new file: `resources/views/mypage.blade.php`


```
<!DOCTYPE html>
<html lang="en">
<head>
  <title>My First Page</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      max-width: 800px;
      margin: 50px auto;
      text-align: center;
    }
    h1 {
      color: #FF2D20;
    }
  </style>
</head>
<body>
  <h1>My First Laravel Page</h1>
  <p>I'm learning Laravel step by step!</p>
  <p>Current date: {{ date('Y-m-d H:i:s') }}</p>
</body>
</html>
```

2. Create a route for it in `routes/web.php`:

```
Route::get('/mypage', function () {
    return view('mypage');
});
```

3. Visit `http://localhost:8000/mypage`

□ Summary

In this lesson, you learned:

- What Laravel is and why it's popular
- Laravel project structure and key folders
- How the request lifecycle works
- How to use Artisan commands
- How to create basic routes
- How to create simple Blade views

▢ Additional Resources

- [Official Laravel Documentation](#)
 - [Laravel News](#)
 - [Laracasts](#) - Video tutorials
-

▢ Test Yourself

Before moving to Lesson 2, make sure you can answer:

1. What is the MVC pattern?
 2. Which file is the entry point for all Laravel requests?
 3. Where are routes defined in Laravel?
 4. What is Artisan?
 5. What is Blade?
-

Next Lesson

Ready for more? Move on to [Lesson 2: Routing Basics](#)

In Lesson 2, you'll learn:

- Different types of routes (GET, POST, PUT, DELETE)
 - Route parameters
 - Named routes
 - Route groups
 - And more!
-

Happy Learning! ☐